3 Group 10

An very good report, drawing on extensive considerations of the frequency response.

3.1 Q1

Good use of a Bode plot to consider frequency response. Your analysis was very good, drawing on considerations in terms of the behaviour of individual circuit components as well as the overall filter response. Calculating the cut-off frequency and then decreasing the frequency outside of the range specified supported your arguments very well.

3.2 Q2

Your method for investigating the order of the error is good, although you were finding the maximum value throughout, rather than the maximum absolute value which would have been more representative of the error. You required some means of determining that the gradient of the graph was equal to 2, such as finding the line of best fit and its gradient. There was no comparison between the observed errors for the three integration routines. Additionally, you could have mentioned the surprising result that the step size of use of $1e^{-6}$ resulted in errors of $\mathcal{O}(10^{-6})$ (due to the large constant C in $e = Ch^2$).

3.3 Q3

Very good analysis, drawing on considerations of individual component behaviour as well as filter response. Your calculation of the resonant frequency and Bode plot allowed you to very precisely explain the observed responses.

3.4 Q4

Very good analysis, across a wide range of initial and boundary conditions. Your physical interpretation was good, although needed a bit more thought in one or two places; for example a sinusoidal initial condition has a net temperature of 0 so there is no net change in energy as time progresses. The bonus was done well.

Grade: A*

EE4-08C Numerical analysis coursework

Group 10: Dr. Kristel Fobelets

Imperial College, March 2017

Q1) 26
Q2) 6
Q3) 26
Q4) 28
86
84

Introduction

Modeling systems is a useful requirement for many modern life aspects, from aeroplanes and circuits to financial markets. When these systems are relatively easy, they can be solved using analytical methods which describe their behaviour accurately. However, when they become more complex, numerical methods are used. These methods are based on continuous iterations of mathematical operations that although they do not get to the exact result, they can be very close and their error is reduced by the number of iterations. Since this process is slow and tidious, computers are used. They generate answers to these problems more quickly than any human would do. There is software speciallized for these kind of tasks such as Matlab. It also allows to generate graphs with the results in a very neat and understandable manner.

LR Circuit

The LR circuit that is going to be studied consists of a reistor and an inductor of values 0.5 Ω and 1.5mH. The output of the system is a potential divider and is given by the transfer function:

$$H(j\omega) = \frac{j\omega L}{j\omega L + R}$$

As it can be seen, at low frequencies the gain will be lower but as the frequency increases the system behaves as a buffer. The analitical solution to this system is given by:

$$L\frac{d}{dt}i_L(t) + Ri_L(t) = V_{in}(t)$$

Where the output voltage is:

$$V_{out}(t) = V_{in}(t) - Ri_L(t)$$

The frequency response of the system is shown in figure 1. It can be seen that the corner frequency is around 60 Hz and it will be prooved in the following tests. Moreover, the phase response shown in figure 2 illustrates the variation of the phase of the signal along the frequency spectrum. At low frequencies the phase variation is of $-\frac{\pi}{2}$ but around the corner frequency it increases very quickly and reaches a steady state value of 0. What these results proove is that this system is very good at transmiting high frequency signals. However, from the motor point of view it illustrates how badly the system would work at low frequencies.

Good.

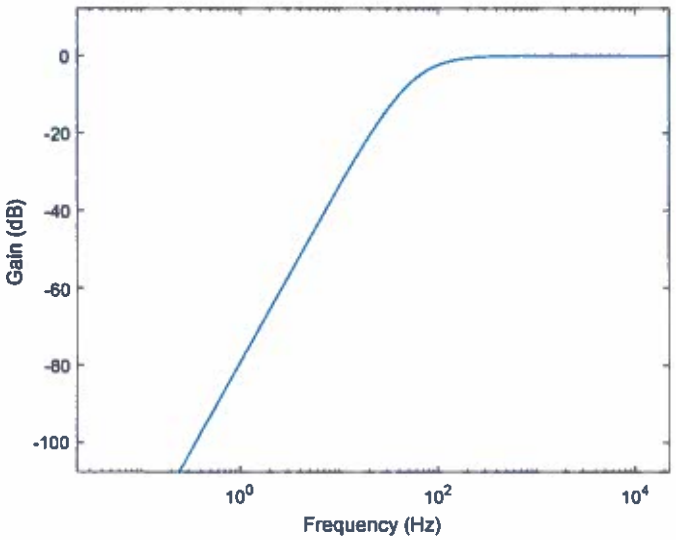


Figure 1: Frequency response of RL circuit

Runge–Kutta methods

Heun

The first method used to model the output voltage of the RL circuit is Heun's method. This is a second order Runge-Kutta method used to solve differential equations. It is an improved version of Euler's method which uses the tangent line of the function at the beginning of the interval to estimate the slope of the function over the interval. Heun's improves on that by using the tangent lines both at the beginning and the end of the interval. One of them is used to predict

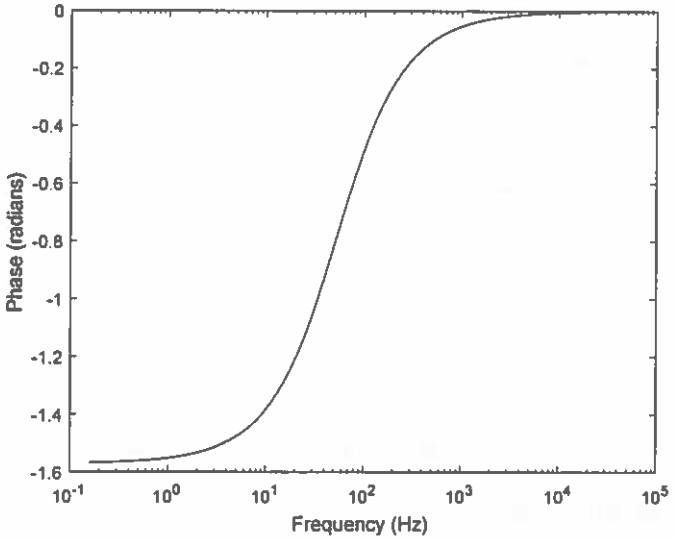


Figure 2: Phase response of RL circuit

and the other one to correct. The average of the two is then taken to calculate the final point.

When given a differential equation of the form $y' = f(x, y)$ we need to break it apart into n points of equal intervals of length l. Using the initial conditions at x_0 and y_0 which are our starting points, we extrapolate further points y_i.

The gradient at x_i, y_i is going to be used for the prediction of the next point using the equation:

$$y_{(p)i+1} = y_i + lf(x_i, y_i)$$

After evaluating that we can use the predicted value of y_{i+1} to predict the gradient at the point x_{i+1}.

$$y'_{i+1} = f(x_{i+1}, y_{i+1})$$

We then use these to calculate the average of the gradients.

$$y'_{(av)} = \frac{y'(i) + y'_{i+1}}{2}$$

$$= \frac{f(x_i, y_i) + f(x_{i+1}, y_{(p)i+1})}{2}$$

Finally, to estimate our "improved" of y_{i+1} value we do $y_{i+1} = y_i + l y'_{(av)}$. The corresponding value of x_{i+1} is simply $x_i + l$, larger by the step that we initialy chose.

Heun's method has been used to implement a Matlab script to calculate the response of an RL circuit given an input voltage, step size, final time and values for R and L which it uses to estimate the output voltage.

The differential equation solved is:

$$\frac{dI}{dt} = \frac{V_{in}(t) - IR}{L}$$

The code is below:

```
% File: heun.m
function [t, Vout] = heun(R, L, Vin, I_0, h, t_final)
    % Determine how many times to step forward
    N = round(t_final/h);

    % Initialise some arrays for the output
    I=zeros(1,N);
    t=zeros(1,N);
    I(1) = I_0;
    Vout(1) = Vin(0) - R*I_0;

    % Represent dI/dt (dI_dt) as a function of time (given R and L)
    dI_dt = @(t, I) (Vin(t) - I*R)/L;

    % Loop as many times as h allows and:
    % calculate the gradient at point
    % Use it to emtimate next two points

    for j=1:N-1 % loop N-1 times
        grad = feval(dI_dt, t(j), I(j));
        I(j+1)=I(j) + h/2*(grad + feval(dI_dt, t(j)+h, I(j)+h*grad));
        t(j+1) = t(j) + h;
        Vout(j+1) = Vin(t(j+1)) - R*I(j+1);
    end
```

Midpoint

The midpoint method, similarly to the Heun's method, is a second order Runge-Kutta method that, but uses a different approach. We start by calculating the gradient at a known point x_i using as $k_1 = f(x_i, y_i)$. After that we calculate

the gradient at a point $x_i + \frac{1}{2}l$ where l is the interval between the points. $k_2 = f(x_i + \frac{1}{2}l, y_+ \frac{1}{2}lk_1)$.

Finally we can estimate the next point as $y_{i+1} = y_i + lk_2$ using the midpoint gradient k_2.

Similarly to heun we can take input parameters in a Matlab function to approximate the output voltage for the RL circuit.

```
% File: midpoint.m
function [t, Vout] = ralston(R, L, Vin, I_0, h, t_final)
    % Determine how many times to step forward
    N = round(t_final/h);

    % Initialise some arrays for the output
    I=zeros(1,N);
    t=zeros(1,N);
    I(1) = I_0;
    Vout(1) = Vin(0) - R*I_0;

    % Represent dI/dt (dI_dt) as a function of time (given R and L)
    dI_dt = @(t, I) (Vin(t) - I*R)/L;

    % Loop as many times as h allows and:
    % calculate the gradient at point
    % Use it to emtimate next two points

    for j=1:N-1 % loop N-1 times
        grad = feval(dI_dt, t(j), I(j));
        I(j+1)=I(j) + h*feval(dI_dt, t(j)+h/2, I(j)+h/2*grad);
        t(j+1) = t(j) + h;
        Vout(j+1) = Vin(t(j+1)) - R*I(j+1);
    end
```

Ralston

Ralston is yet another second order Runge-Kutta method for solving differential equations and is implemented according to the following steps. We first calculate the gradient at a point x_i as $k_1 = lf(x_i, y_i)$.

Using this gradient we can calculate the gradient at a point $x_i + \frac{2}{3}l$.

$$k_2 = lf(x_i + \frac{2}{3}l, y_i + \frac{2}{3}k_1).$$

Finally in order to estimate y_{i+1} we use $y_{i+1} = y_i + \frac{1}{4}k_1 + \frac{3}{4}k_2$.

Similarly to the Heun's and the midpoint method we can implement this in Matlab, taking input arguments. In the next section we will try varying the input parameters, in particular the inpunt voltage, and see what kind of output is produced from the circuit.

```
function [t, Vout] = ralston(R, L, Vin, I_0, h, t_final)
    % Determine how many times to step forward
    N = round(t_final/h);

    % Initialise some arrays for the output
    I=zeros(1,N);
    t=zeros(1,N);
    I(1) = I_0;
    Vout(1) = Vin(0) - R*I_0;

    % Represent dI/dt (dI_dt) as a function of time (given R and L)
    dI_dt = @(t, I) (Vin(t) - I*R)/L;

    % Loop as many times as h allows and:
    % calculate the gradient at point
    % Use it to emtimate next two points

    for j=1:N-1 % loop N-1 times
        grad = feval(dI_dt, t(j), I(j));
        I(j+1)=I(j) + h/4*(grad + 3*feval(dI_dt, t(j)+2/3*h, I(j)+2/3*h*grad));
        t(j+1) = t(j) + h;
        Vout(j+1) = Vin(t(j+1)) - R*I(j+1);
    end
```

Input Functions

The Matlab function implementing Ralston's method can be given any voltage input as a parameter which it will then use to estimate the output voltage response of the system. We generate the graphs with a second script.

```
% File: ralston_script.m
% Vin = aI' + bI

R = 0.5;    % 0.5Ohm
L = 0.0015; % 1.5mH
data_points = 10000;

% Go on for 8 time constants
time_constant = L/R;
step = time_constant*8/data_points;
```

```
11
12  % Signal 1 Vin = 5.5V
13  Vin = @(t) 5.5 + 0*t;  % 5.5V
14  current_initial=0;
15
16  [t, Vout] = ralston(R, L, Vin, current_initial, step, step*data_points);
17  figure;
18  plot(t,5.5*heaviside(t)); %plot input
19  hold on;
20  plot(t, Vout);
21  xlabel('Time (s)') % x-axis label
22  ylabel('Amplitude (V)') % y-axis label
23
24  legend('Input','Output')
25
26
27  % Signal 2a Vin = 3.5*exp(-t^2/τ) V
28  step = time_constant*20/data_points;
29  Vin = @(t) 3.5*exp(-t^2/(160e-6));  % 3.5V
30  current_initial=0;
31
32  [t, Vout] = ralston(R, L, Vin, current_initial, step, step*data_points);
33  figure;
34  plot(t, 3.5*exp(-t.^2/(160e-6))); %plot input
35  hold on;
36  plot(t, Vout);
37  xlabel('Time (s)') % x-axis label
38  ylabel('Amplitude (V)') % y-axis label
39  legend('Input','Output')
40
41  % Signal 2b Vin = 3.5*exp(-t/τ)
42  step = time_constant*2/data_points;
43  Vin = @(t) 3.5*exp(-t/(160e-6));  % 3.5V
44  current_initial=0;
45
46  [t, Vout] = ralston(R, L, Vin, current_initial, step, step*data_points);
47  figure;
48  plot(t,3.5*exp(-t/(160e-6))); % plot input
49  hold on;
50  plot(t, Vout);
51  xlabel('Time (s)') % x-axis label
52  ylabel('Amplitude (V)') % y-axis label
53  legend('Input','Output')
54
55
56  % Signal 3
```

Printer did not print page 8.
Please refer to online copy. (code)

Printed copy (from labs downstairs) contains minor artifacts not present in PDF.

- some headings have wrong font
- missing function bracket {
- missing page 8 (only code)

Please use submitted PDF on blackboard as reference.

Thank you,
Group 10 Dr. Kr. Fobelets

```
plot(t, Vout);
xlabel('Time (s)') % x-axis label
ylabel('Amplitude (V)') % y-axis label
legend('Input','Output')

%Square%
Vin = @(t) 4.5*square(2*pi*f*t);
[t, Vout] = ralston(R, L, Vin, current_initial, step, data_points*step);
figure;
plot(t,4.5*square(2*pi*f*t)); %plot input
hold on;
plot(t, Vout);
xlabel('Time (s)') % x-axis label
ylabel('Amplitude (V)') % y-axis label
legend('Input','Output')

%Sawtooth%
Vin = @(t) 4.5*sawtooth(2*pi*f*t);
[t, Vout] = ralston(R, L, Vin, current_initial, step, data_points*step);
figure;
plot(t,4.5*sawtooth(2*pi*f*t)); %plot input
hold on;
plot(t, Vout);
xlabel('Time (s)') % x-axis label
ylabel('Amplitude (V)') % y-axis label
legend('Input','Output')


%T = 450us %
%Sine%
f = 1/450e-6; % 450us
step = time_constant*0.8/data_points;
Vin = @(t) 4.5*sin(2*pi*f*t);
[t, Vout] = ralston(R, L, Vin, current_initial, step, data_points*step);
figure;
plot(t,4.5*sin(2*pi*f*t)); %plot input
hold on;
plot(t, Vout);
xlabel('Time (s)') % x-axis label
ylabel('Amplitude (V)') % y-axis label
legend('Input','Output')

%Square%
Vin = @(t) 4.5*square(2*pi*f*t);
[t, Vout] = ralston(R, L, Vin, current_initial, step, data_points*step);
figure;
```

```
plot(t,4.5*square(2*pi*f*t)); %plot input
hold on;
plot(t, Vout);
xlabel('Time (s)') % x-axis label
ylabel('Amplitude (V)') % y-axis label
legend('Input','Output')

%Sawtooth%
Vin = @(t) 4.5*sawtooth(2*pi*f*t);
[t, Vout] = ralston(R, L, Vin, current_initial, step, data_points*step);
figure;
plot(t,4.5*sawtooth(2*pi*f*t)); %plot input
hold on;
plot(t, Vout);
xlabel('Time (s)') % x-axis label
ylabel('Amplitude (V)') % y-axis label
legend('Input','Output')


%T = 1007us %
%Sine%
f = 1/1000e-6;% 1000us
step = time_constant*0.8/data_points;
Vin = @(t) 4.5*sin(2*pi*f*t);
[t, Vout] = ralston(R, L, Vin, current_initial, step, data_points*step);
figure;
plot(t,4.5*sin(2*pi*f*t)); %plot input
hold on;
plot(t, Vout);
xlabel('Time (s)') % x-axis label
ylabel('Amplitude (V)') % y-axis label
legend('Input','Output')

%Square%
Vin = @(t) 4.5*square(2*pi*f*t);
[t, Vout] = ralston(R, L, Vin, current_initial, step, data_points*step);
figure;
plot(t,4.5*square(2*pi*f*t)); %plot input
hold on;
plot(t, Vout);
xlabel('Time (s)') % x-axis label
ylabel('Amplitude (V)') % y-axis label
legend('Input','Output')

%Sawtooth%
Vin = @(t) 4.5*sawtooth(2*pi*f*t);
```

```
[t, Vout] = ralston(R, L, Vin, current_initial, step, data_points*step);
figure;
plot(t,4.5*sawtooth(2*pi*f*t)); %plot input
hold on;
plot(t, Vout);
xlabel('Time (s)') % x-axis label
ylabel('Amplitude (V)') % y-axis label
legend('Input','Output')
```

Step Input

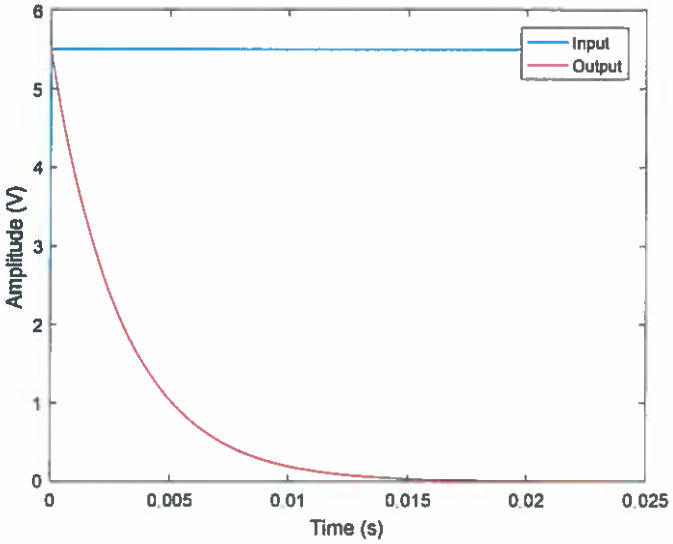


Figure 3: Step Response

$$(1)V_{in} = 5.5V$$

In the response we see the output starting off at 5.5V and quickly decaying towards 0.

Before we switch on the input, the current in the inductor is 0. As the inductor current cannot change instantly, it will act as an open circuit at $t(0^+)$ taking up the entire input voltage. After activation, as time increases the inductor current will increase and the voltage drop across the resistor will increase. As time approaches infinity, the current through the resistor will approach $\frac{V}{R}$ and the voltage drop across the inductor (our output voltage) will reach 0. Another way

to think about this is that the inductor becomes a short circuit at DC therefore connecting the output to ground. This is the principle of a high pass filter, which eliminates the DC component. The decay as the current increases can be seen in figure 3.

From a mathematical point of view, the derivative of the current is high at $t(0^+)$ but it decays as time increases and so does the voltage.

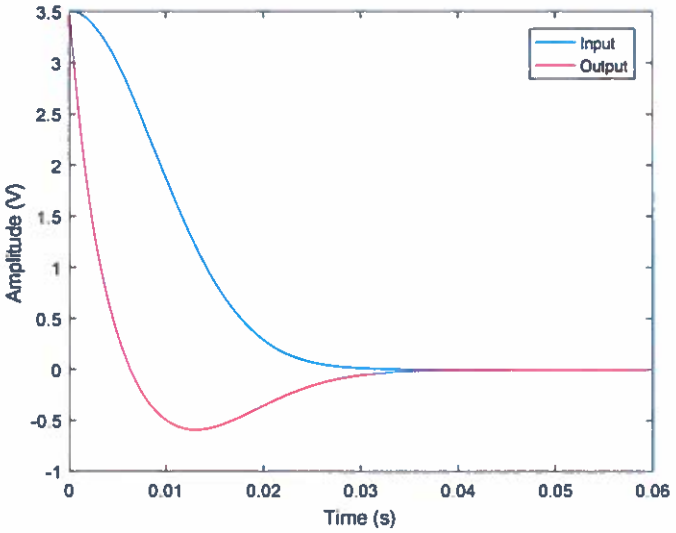


Figure 4: Impulsive Signal and Decay - 160(us)^2

Impulsive Input

$$V_{in} = 5.5e^{\frac{-t^2}{\tau}}$$

$$V_{in} = 5.5e^{\frac{-t}{\tau}}$$

The input voltages in figures 4 and 5 are exponentially decaying functions with time constants of $160us$ and $160(us)^2$. Every signal has frequency components that can be analyzed using the fourier transform. Signals that have higher rate of change have higher frequency components and this effect can be obseerved by comparing figures 4 and 5. The time constant of the first impulse is larger than the one of the second and therefore it decays more quickly. Since the circuit is a high pass filter, some of the components of the signal will not be reduced as much as the fundamental frequency component of the impulse. It is because of this reason that while the first impulse has some oscillation before reaching a

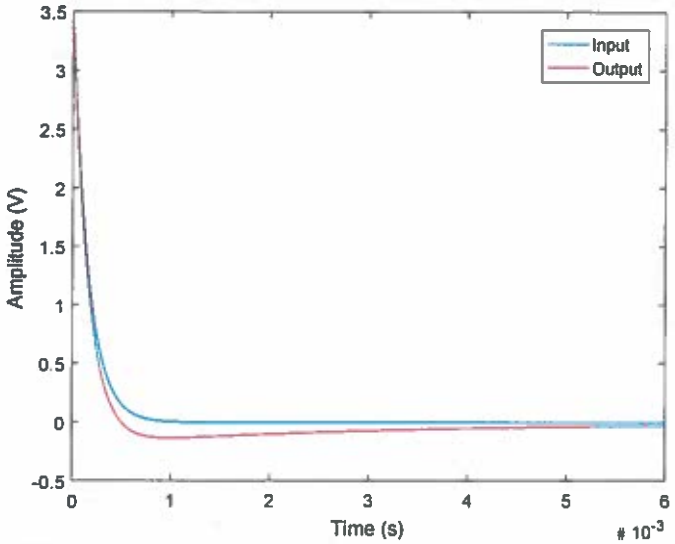


Figure 5: Impulsive Signal and Decay - 160us

steady state of 0 V, the second impulse has a high damping factor and does not oscillate.

Sine, Square and Sawtooth Wave Input

The circuit we have is a RL high-pass filter. To calculate the cut-off frequency of the filter, we use $\frac{R}{2\pi L}$, which in this case gives us a cut-off frequency of approximately 53Hz.

good

Sine Wave Inputs

In this set of inputs a sinusoid of different frequencies and amplitude 4.5V will be set as the input of the system. It will come to our understanding how the phase and the gain vary as we increase the frequency of the mentioned signal. The circuit we have is an RL high-pass filter. To calculate the cut-off frequency of the filter, $\frac{R}{2\pi L}$ is used, which in this case results in a cut-off frequency of approximately 53Hz.

The first sinusoid has a frequency of 50,000 Hz as shown in figure 6. This is very much above the corner frequency of 53 Hz. It is because of this that output signal will be very close to the input signal since the gain at infinity is unity

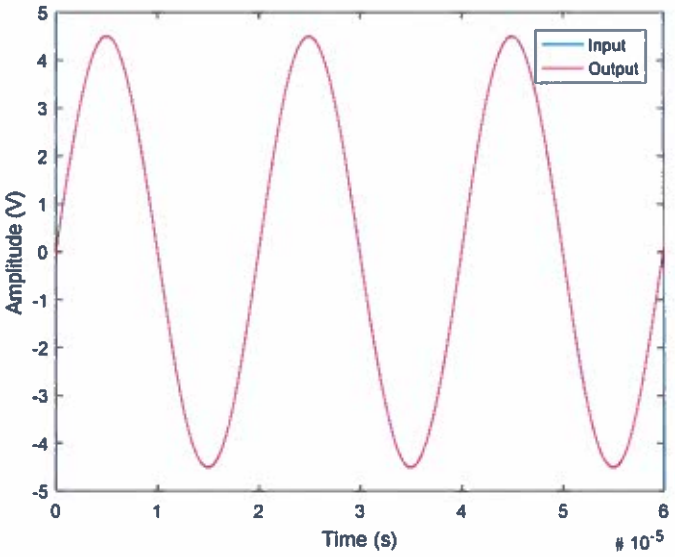


Figure 6: 20us Sine Wave Input

and the phase difference is 0. This system becomes at high frequencies a very accurate buffer.

The following signal as seen in figure 7 has a frequency of 6250 Hz. The output illustrates a similar behaviour to that seen in figure 6.

When a sinusoid of 2200 Hz is given as an input to the system as in figure 8, it can be seen how the amplitude starts to decay and it also seems to show a slight phase difference. This is due to the fact that the frequency of the signal is approaching the corner frequency and therefore, the gain starts to become smaller and the output starts to lag the input.

Finally, in figure 9, the frequency of the sinusoid becomes 1000 Hz, it is now approaching the corner frequency. The circuit now has a noticable effect on the input. The output has lower amplitude than the input and it lags it by a small degree.

✓ v. good

Square Wave Inputs

In figure 11, figure 12 and figure 13, the DC component of the square wave is filtered by the RL circuit. The output should then reflect the curve observed in the step input of 3. The curve appears linear as the DC component occurs in a small period of time, therefore the curve can be modelled to be mostly linear.

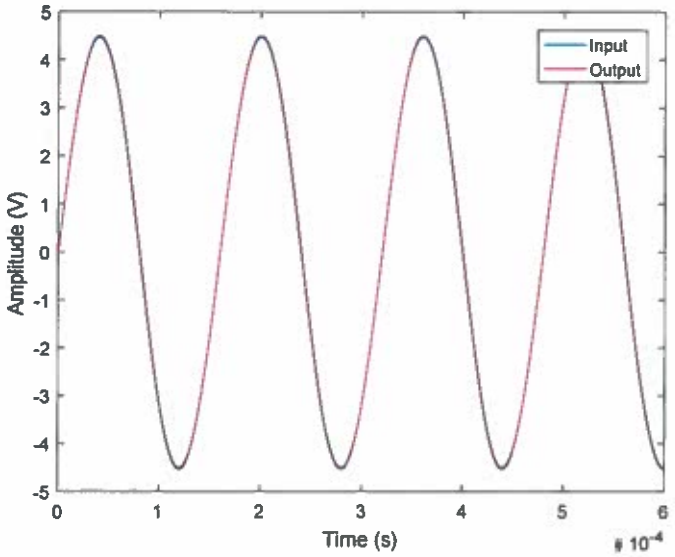


Figure 7: 160us Sine Wave Input

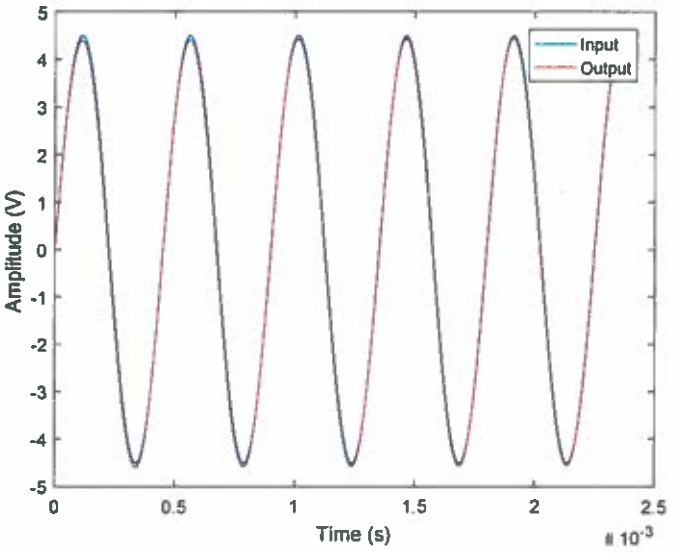


Figure 8: 450us Sine Wave Input

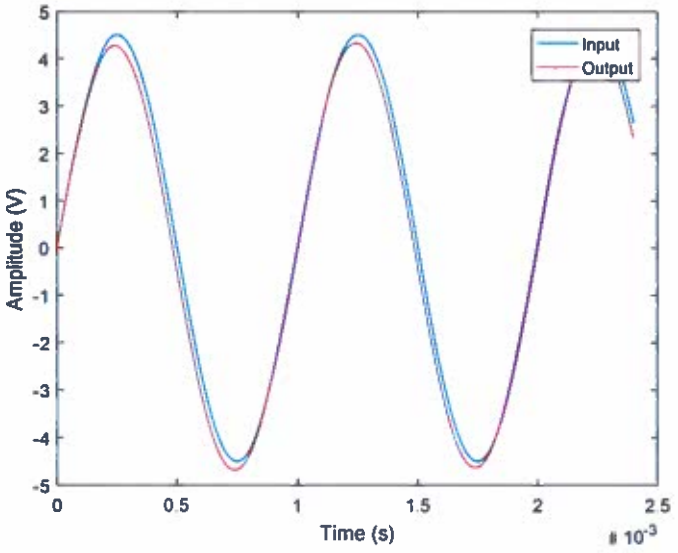


Figure 9: 1000us Sine Wave Input

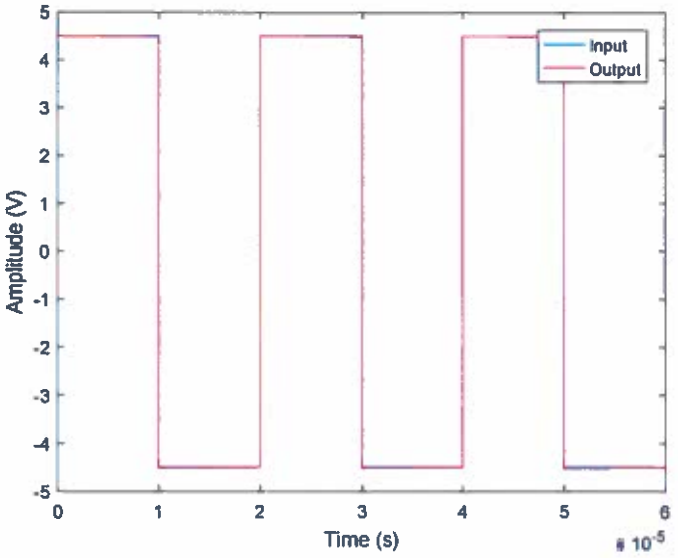


Figure 10: 20us Square Wave response

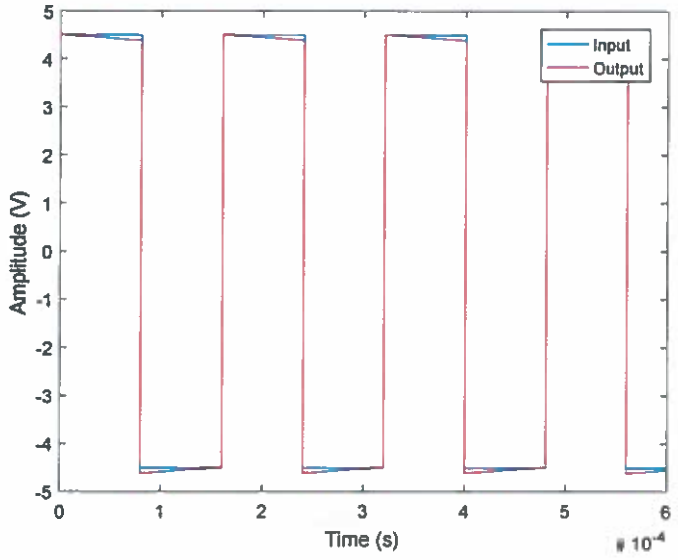


Figure 11: 160us Square Wave response

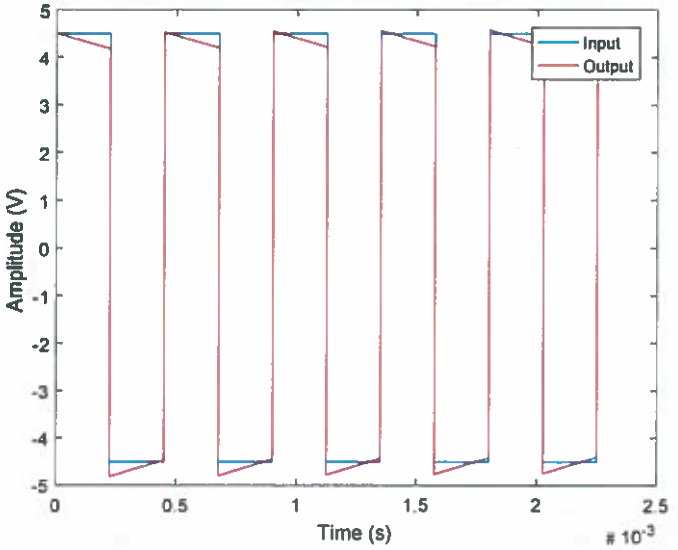


Figure 12: 450us Square Wave response

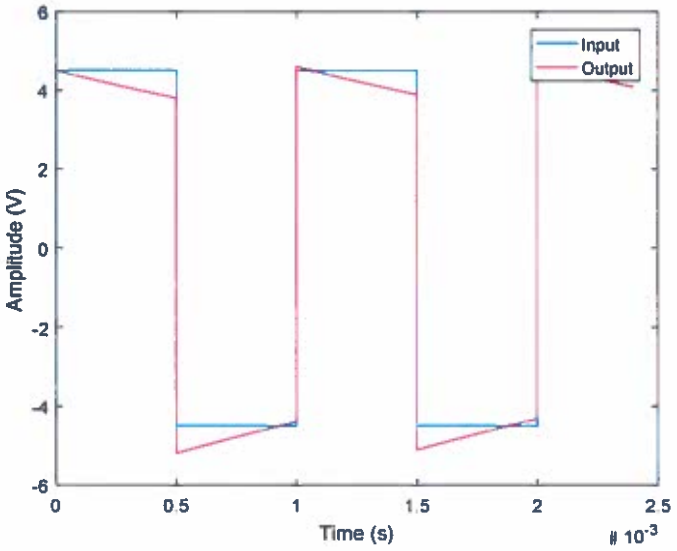


Figure 13: 1000us Square Wave response

If the frequency of the square wave was decreased further, the similarity of the curve in 3 and the output voltage would be significantly more noticeable. However, for figure 10 there is no obvious loss of the DC component. This is because the frequency of the input square wave is too high, such that the filtered output cannot decrease or increase rapidly enough to reflect the loss of the DC component.

Sawtooth Wave Inputs

In figure 17, we see a slight curve in the output relative to the positive gradient of the input sawtooth wave. This is because the RL circuit acts as a differentiator, but is not given enough time to increase to the steady state value it wants to achieve. However, in figure 14, figure 15 and figure 16, the frequencies of the inputs are too high, giving the output even less time to reach the desired steady state value. The output waves then look like sawtooth functions with a linear increase.

If we generate a graph with a lower frequency triangle wave input, we would get an output similar to a square wave, as seen in figure 18. The nature of the circuit as a differentiate becomes much more evident, as the derivative of a triangle wave is indeed a square wave, as the slope of the triagnle wave is constant and hence is the amplitude of the derivative.

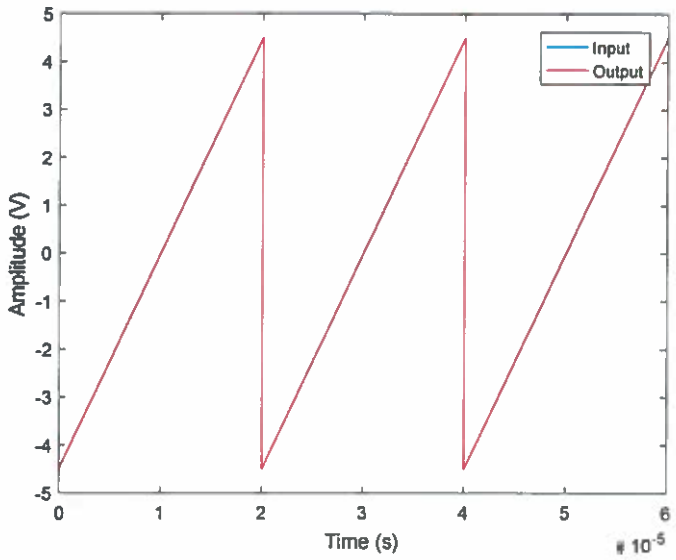


Figure 14: 20us Sawtooth Wave response

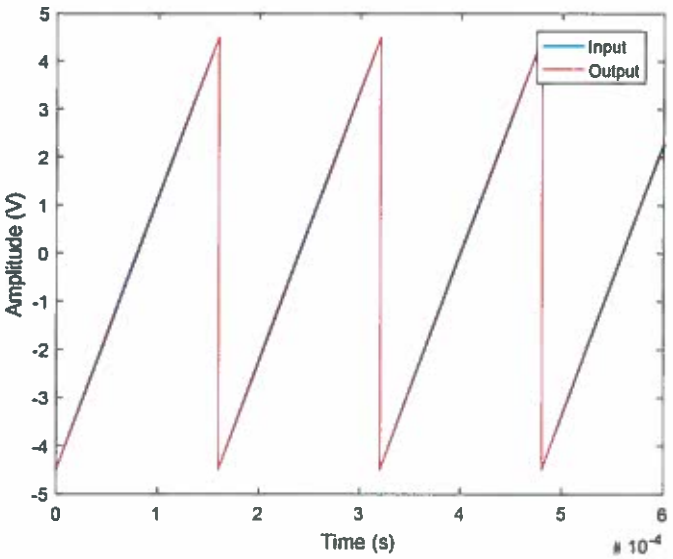


Figure 15: 160us Sawtooth Wave response

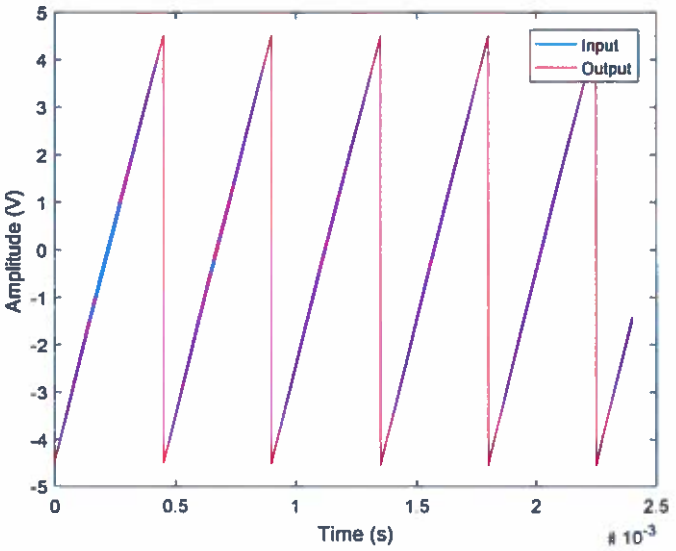


Figure 16: 450us Sawtooth Wave response

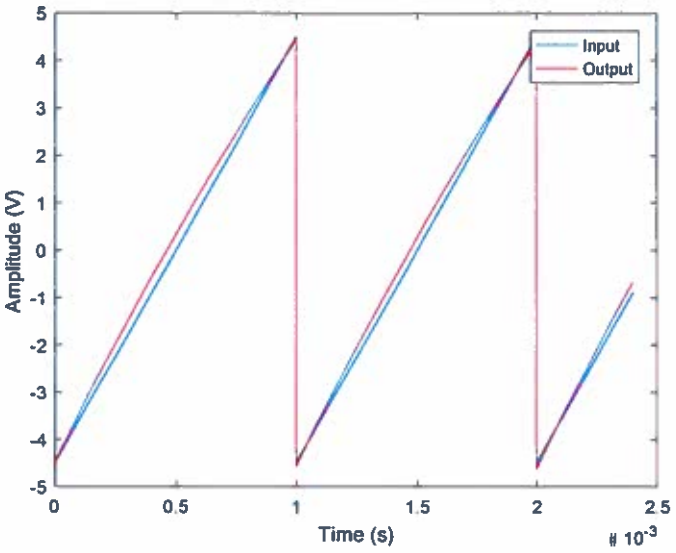


Figure 17: 1000us Sawtooth Wave response

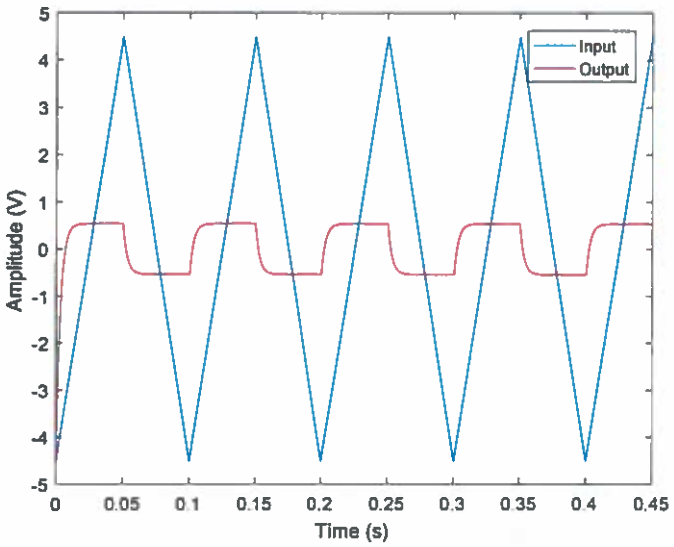


Figure 18: 100ms Triangle Wave response

Error Analysis

Error analysis is performed with error_script.m which simply generates appropriate arguments to give to ralston.m, and then runs it in a loop in which it sets the step size as the product of the original step and the iterator. Inside the loop it compares every value returned in the array by ralston.m with the exact value that one can obtain by solving the circuit by using the integrating factor. It takes the maximum error for each time step and saves it to a new array which is then plotted. The code is as follows:

```
% File: error_script.m
% This script will carry out error analysis
R = 0.5;     % 0.5Ohm
L = 0.0015; % 1.5mH

% How many data points we want to have for each step size
data_points = 100000;

% How many step sizes to try
h_divisions = 1000;

% Go on for a time constant
time_constant = L/R;
step = time_constant/data_points;
```

```
% Define input parmeters

T=150e-6; % 150us
f = 1/T;
w = 2*pi*f;
A = 6;

% Input voltage function
Vin = @(t) A*cos(w*t);

% Initial current is zero
I0=0;


% Define coefficients found from exact solution
A = 6*R/(R^2 + (w*L)^2);
B = 6*w*L/(R^2 + (w*L)^2);


% Exact current as founnd using integrating factor
current_exact = @(t) A*cos(w*t) + B*sin(w*t) - A*exp((-R/L)*t);
Vout_exact = @(t) Vin(t) - current_exact(t)*R;


% For each time step run ralston funcntion
% and compare each data point to the exact solution
% save the mamum error of each time step in an array

for k=1:h_divisions
    [t, Vout] = midpoint(R, L, Vin, I0, step*k, data_points*step);

    for j=1:data_points/k
        error_array(j) = Vout_exact(t(j)) - Vout(j);
    end

    max_array(k) = max(error_array);
    max_t(k) = k*step;
end

% Plot on log-log plot the step size vs error

loglog(max_t, max_array);
title('Order of error')
xlabel('Time step size')
ylabel('Maximum error')
```

not absolute value

The resulting plot can be seen in figure 19. Take note that the axis of the graph are both logarithmic! We observe that the slope of the graph is 2. This means that the error is a quadratic function of the step size. Ralston is a second order Runge-Kutta method, which means that the location truncation error is of order $O(h^3)$, while the total accumulated error is of order $O(h^2)$ and this agrees with our results from figure 19.

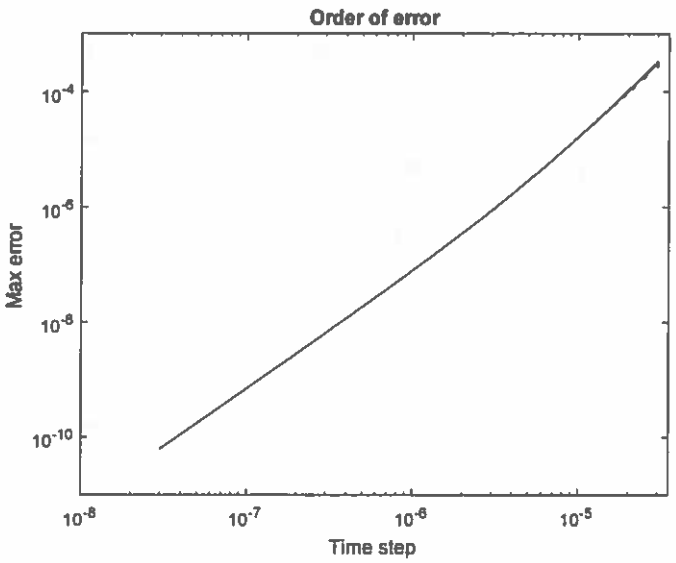


Figure 19: Error of Ralston approximation

LRC circuit

Introduction

During this exercise the behaviour of an RLC circuit was studied. As it can be seen in figure 20, a signal is applied to a circuit with an inductor followed by a resistor and then a capacitor. The output voltage is then measured at both ends of the resistor. As it will be shown the circuit behaves as a band-pass filter. The equation that models the system is:

$$L\frac{d^2}{dt^2}q_c(t) + R\frac{d}{dt}q_c(t) + \frac{1}{C}q_c(t) = V_{in}(t)$$

With the resistor, capacitor and inductor values being 280 Ω, 4 μF and 600 mH,

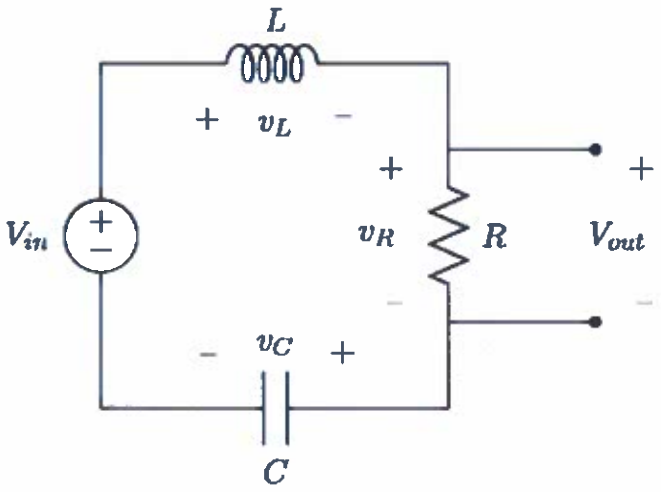


Figure 20: Circuit diagram. Credit: Dr. Daniel Nucinkis

respectively. The frequency response of the circuit is the one shown in figure 21 and it's given by:

$$H(j\omega) = \frac{j\omega RC}{1 + j\omega RC + (j\omega)^2 LC}$$

Figure 21 shows how the filter will block any DC voltage and reduce the gain of low frequency signals. The corner frequency is given by $w_c = \frac{1}{\sqrt{LC}}$ which gives 102.73 Hz. Therefore, the gain will increase as the frequency becomes larger until it reaches that frequency. Here the gain will be maximum and unity. Any larger frequencies will be filtered out.

The damping factor ζ is calculated as $\frac{RC}{\sqrt{4LC}}$ and it gives 0.36. Because this number is lower than 1, it should be appreciated that there is a resonance (in this case at 102.73 Hz).

It can be seen in figure 22 how the phase shifts all over the frequency spectrum. There is almost no change at each of the sides of the corner frequency. However, there is a drastic change around such frequency, from $-\frac{\pi}{2}$ to $\frac{\pi}{2}$.

Implementation

In order to calculate the output of the system a numerical method was used, the Runge-Kutta $\frac{3}{8}$ algorithm[1]. There are infinitely many Runge-Kutta methods.

[1]https://en.wikipedia.org/wiki/Runge%E2%80%93Kutta_methods

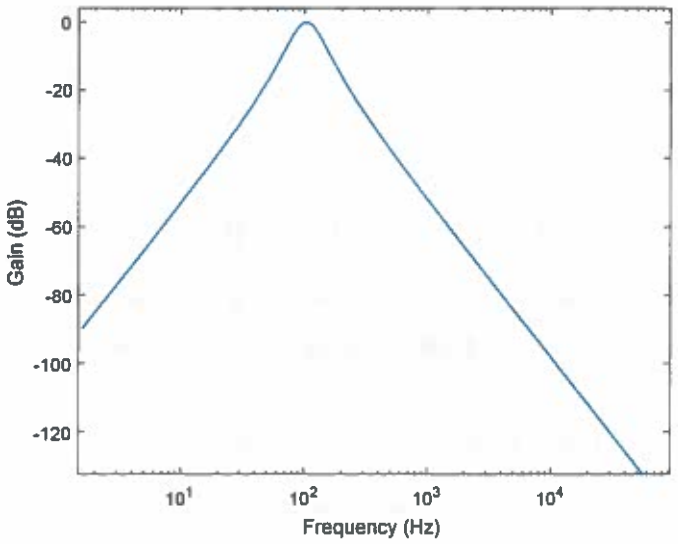


Figure 21: Frequency response

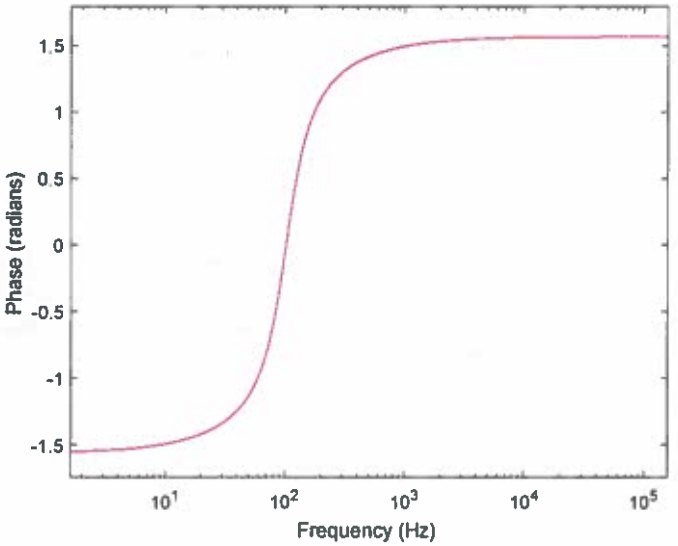


Figure 22: Phase response

However, they all have different coefficients. The ones that were used are shown in figure 23.

0	0	0	0	0
1/3	1/3	0	0	0
2/3	−1/3	1	0	0
1	1	−1	1	0
	1/8	3/8	3/8	1/8

Figure 23: Runge-Kutta matrix for $\frac{3}{8}$ algorithm.

This leads to the construction of the following system of equations for one step calculation:

$$k_1 = hf(x_i, y_i)$$
$$k_2 = hf(x_i + h\frac{1}{3}, y_i + k_1\frac{1}{3})$$
$$k_3 = hf(x_i + h\frac{2}{3}, y_i - k_1\frac{1}{3} + k_2)$$
$$k_4 = hf(x_i + h, y_i + k_1 - k_2 + k_3)$$
$$y_{i+1} = y_i + \frac{1}{8}(k_1 + 3k_2 + 3k_3 + k_4)$$
$$x_{i+1} = x_i + h$$

Where f is the function of the system. Because this is a second order differential equation two coupled first order differential equations had to be used in such way that $y = \frac{dx}{dt}$ and $x(t)$ becomes the charge of the circuit. For the parameters used in the Matlab file these are their values:

$$y = \frac{d}{dx}$$
$$x = \frac{d}{dt}$$

In order to do this two equal algorithms were computed at the same time step using their respective coefficients. The equations now become:

$$k_{1x} = hg(t, x, y)$$
$$k_{1y} = hf(t, x, y)$$
$$k_{2x} = hg(t + h\frac{1}{3}, x_i + k_{1x}\frac{1}{3}, y_i + k_{1y}\frac{1}{3})$$
$$k_{2y} = hf(t + h\frac{1}{3}, x_i + k_{1x}\frac{1}{3}, y_i + k_{1y}\frac{1}{3})$$
$$k_{3x} = hg(t_i + h\frac{2}{3}, x_i - k_{1x}\frac{1}{3} + k_{2x}, y_i - k_{1y}\frac{1}{3} + k_{2y})$$
$$k_{3y} = hf(t_i + h\frac{2}{3}, x_i - k_{1x}\frac{1}{3} + k_{2x}, y_i - k_{1y}\frac{1}{3} + k_{2y})$$
$$k_{4x} = hg(t_i + h, x_i + k_{1x} - k_{2x} + k_{3x}, y_i + k_{1y} - k_{2y} + k_{3y})$$
$$k_{4y} = hf(t_i + h, x_i + k_{1x} - k_{2x} + k_{3x}, y_i + k_{1y} - k_{2y} + k_{3y})$$
$$x_{i+1} = x_i + \frac{1}{8}(k_{1x} + 3k_{2x} + 3k_{3x} + k_{4x})$$
$$y_{i+1} = y_i + \frac{1}{8}(k_{1y} + 3k_{2y} + 3k_{3y} + k_{4y})$$

Two matlab files are used. RLC_script.m has all the input parameters such as initial conditions, values of the components and functions of the system. As it can be seen, RLC_script.m runs a loop that increases time by a small step size and calculates the differential equation one ahead of the previous value. The smaller the step size h the smaller the error but also larger the computation time. The functions g and f are used and they represent the X' and Y' functions of the system, respectively.

```
1  h = 1e-4;                                  %Step size
2  R = 280;                                   %Resistor value
3  C = 4*10^(-6);                             %Capacitor value
4  L = 600*10^(-3);                           %Inductor value
5  t0 = 0;                                    %Initial time
6  tf = 0.4;                                  %Final time
7  N = round((tf-t0)/h);                      %Number of steps
8  y = zeros(1, N);                           %Preallocate space for efficiency
9  x = zeros(1, N);
10 t = zeros(1, N);
11 Vout = zeros(1, N);
12 Input = zeros(1, N);
13 t(1) = t0;                                 %Initialize the time array
14 x(1) = 500*10^(-9);                        %Initial condition
15 y(1) = 0;                                  %Initial condition
16 %Vin = @(t)5*square(5*2*pi*t);
17 %Vin = @(t)5*sin(500*2*pi*t);
18 %Vin = @(t)5;
```

```
%Vin = @(t)5*exp(-t.^2/0.003);          %Input to the system
g = @(t,x,y) y;                         %Function 1
f = @(t,x,y)(Vin(t) - x/C - R*y)/L;     %Function 2
m = 1;
for i = 1:N
    Input(i) = Vin(t(i));
    [x(i+1), y(i+1)] = RK4second(t(i),x(i),y(i),h,g,f); %Call RK4second.m
    t(i+1) = t(i) + h;                  %Increase time
    Vout(i) = R*y(i);                   %Calculate Vout
end
t(N+1)=[];                  %t vector > Vout & Vin vectors, delete last
figure(1);                  %element
plot(t,Vout,'blue');        %Plot Vout
xlabel('Time(s)');
ylabel('Amplitude(V)');
hold on;
plot(t,Input,'red');        %Plot input
xlabel('Time(s)');
ylabel('Amplitude(V)');
legend('Vout', 'Vin');
```

Every time the loop is executed, a function RK4second is called This Matlab algorithm uses the previous values of x and y for a given time and returns the next values of x and y. In this function the coefficients of the Runge-Kutta method are calculated. When it returns to the main file time is updated ready to calculate the next step and Vout is computed as $R\frac{dx}{dt}$.

```
function [x, y] = RK4second(t,x,y,h,g,f)
k1x = h*g(t, x,y);                              %Step 1 for x
k1y = h*f(t, x,y);                              %Step 1 for y
k2x = h*g(t+h/3, x+k1x/3, y+k1y/3);             %Step 2 for x
k2y = h*f(t+h/3, x+k1x/3, y+k1y/3);             %Step 2 for y
k3x = h*g(t+2*h/3, x-k1x/3+k2x, y-k1y/3+k2y);   %Step 3 for x
k3y = h*f(t+2*h/3, x-k1x/3+k2x, y-k1y/3+k2y);   %Step 3 for y
k4x = h*g(t+h, x+k1x-k2x+k3x, y+k1y-k2y+k3y);   %Step 4 for x
k4y = h*f(t+h, x+k1x-k2x+k3x, y+k1y-k2y+k3y);   %Step 4 for y
x = x+(1/8)*(k1x+3*k2x+3*k3x+k4x);              %Update x
y = y+(1/8)*(k1y+3*k2y+3*k3y+k4y);              %Update y
end
```

Input tests

Input = 5V

Figure 24 shows a DC input of 5V inputed into the system. Because of the frequency response shown in figure 21 it can be seen that any DC signal will

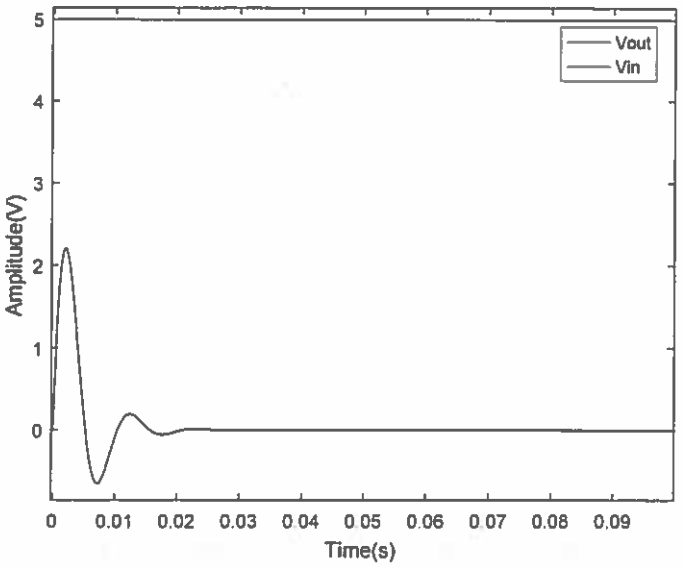


Figure 24: 5V DC input

be blocked. Moreover, the steady state for an inductor is short circuit and for a capacitor is open circuit. Since there will be no current flowing through the resistor, there will not be a voltage drop and therefore the output voltage will be zero. The initial transient is due to the infinite derivative of the current and voltage. Since at $t = 0$ it's behaving as a square wave, the change of voltage will allow current to flow through the circuit but it does not last long since the derivative goes to zero immediately after. The initial condition of the system affects the amplitude of the transient as it will be seen with other inputs.

good

Exponential input

The input signal seen in figure 25 is given by $5e^{\frac{t^2}{0.09}}V$ Because the frequency components of the decaying signal are mainly low frequency it can be observed that the system reduces the gain of the signal and it almost looks like the output from figure 24. However, after the first transient at the output there is a small oscillation because there is still a signal at the input.

Input = 5 V 5Hz sine wave

The input of figure 26 is a very low frequency sine wave of 5 Hz. Taking a look again at figure 21 it is clearly seen that the gain at such frequency is very low. Moreover, as shown in figure figure 22 the phase difference is of $-\frac{\pi}{2}$.

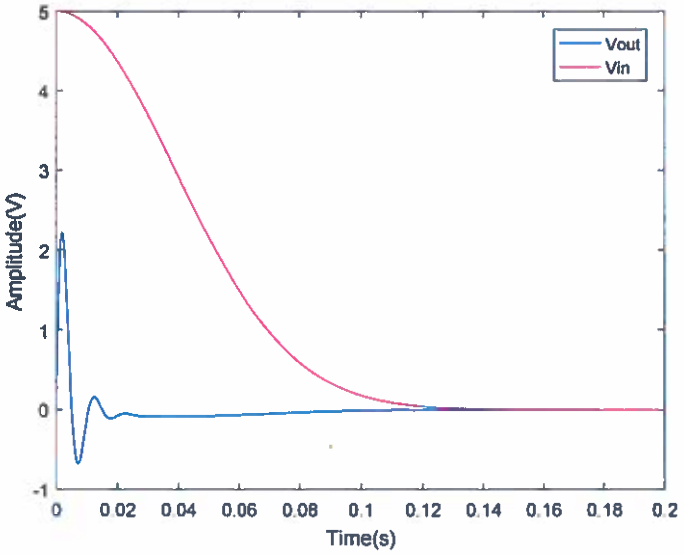


Figure 25: Exponential input

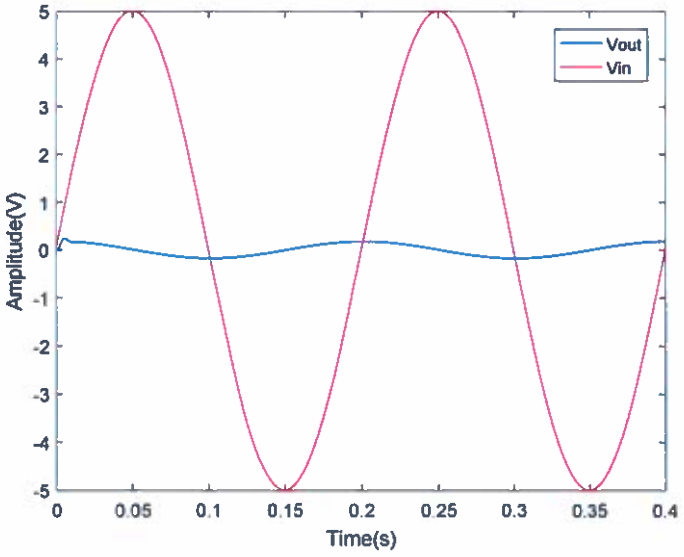


Figure 26: 5 Hz sinc wave

Input = 5V 110 Hz sine wave

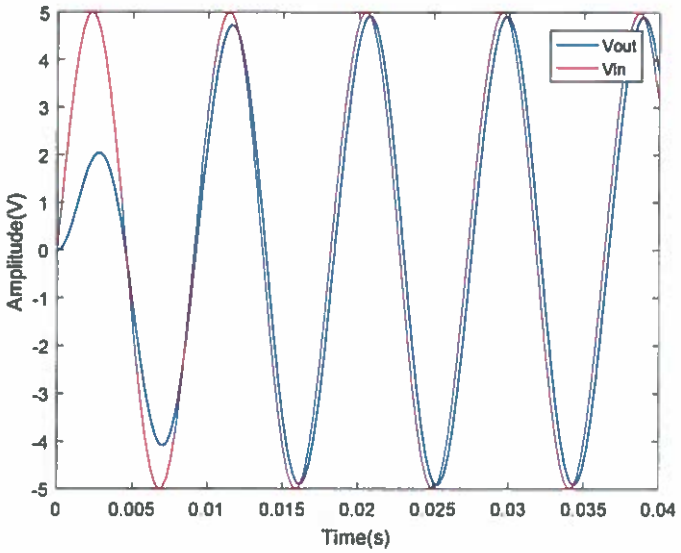


Figure 27: 110 Hz sine wave

At this frequency the sinusoid is very close to the resonant frequency. Here the gain is maximum and equal to unity. The phase is also very close to zero. It is because of this that the input is very similar to the output and it makes this system very suitable for transmitting signals around this frequency.

Input = 5 V 500 Hz sine wave

At higher frequencies the gain goes down again and the phase aproaches $fracπ2$ as shown in figure 28. The oscillation at the beginning is due to the same phenomena seen in figure 24.

Input = 5 V 5 Hz square wave

The waveform you can seee in figure 29 of the input is now a square wave of low frequency, 5Hz. What the output sees is a periodic 5V DC input as in figure 24 and inverted when the voltage goes to -5V. The reason for the first transient being smaller than the rest is the effect of the initial condition that decreases the amplitude because of making the derivative smaller.

Input = 5 V 110 Hz square wave

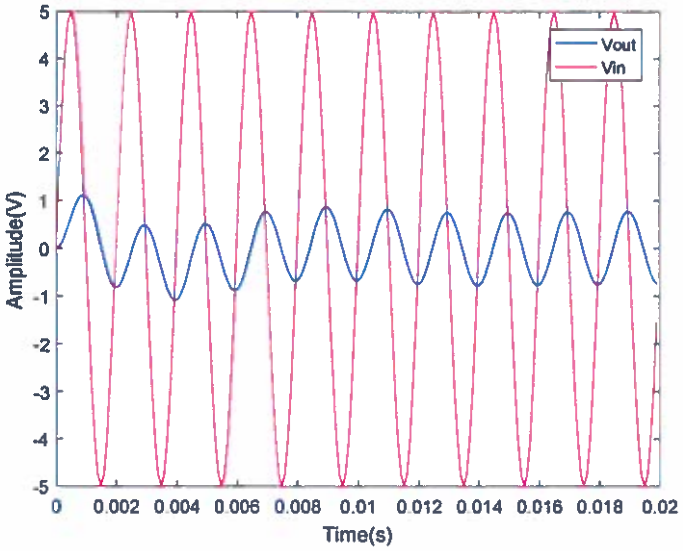


Figure 28: 500Hz sine wave

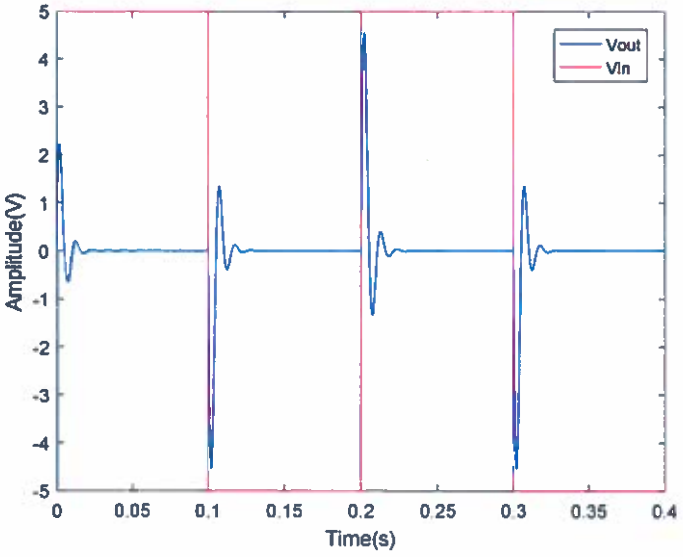


Figure 29: 5Hz square wave

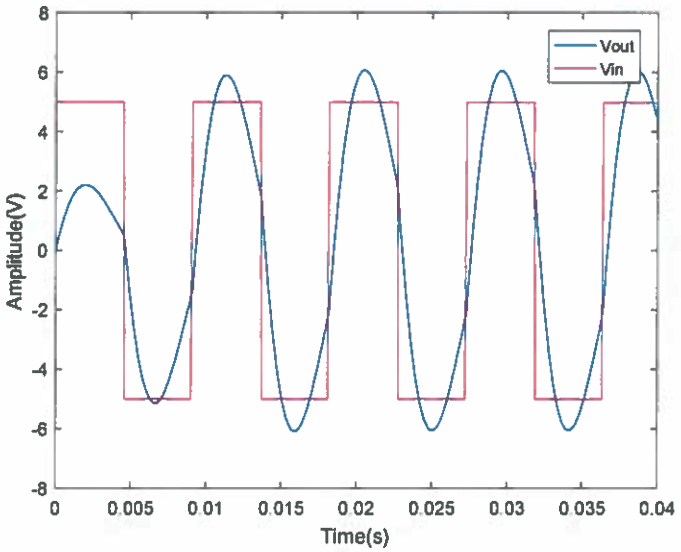


Figure 30: 110Hz square wave

The square wave is now near the resonant frequency. Square waves have a large frequency spectrum. What can be observed from figure 30 is how the circuit blocks most of the frequency components away from the fundamental frequency (110 Hz). Therefore, the output is a slightly distorted sinusoid because the nearby components haven't been reduced completely.

v. good.

Input = 5 V 500 Hz square wave

A much larger frequency square wave is shown in figure 31. The only frequency components that are not filtered out form a triangle wave. It is also observable how the gain is severely reduced due to the frequency response.

Finite Differences for PDE

Implementation:

We first create a matrix to implement our finite difference method on. The two dimensions of the matrix are time and displacement.

The first row of the matrix (in other words at time $t = 0$) is populated with the initial conditions.

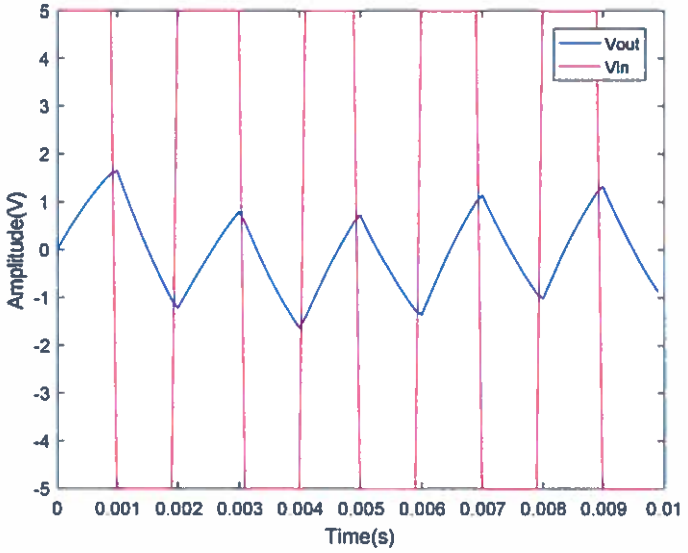


Figure 31: 500Hz square wave

The rest of the matrix is populated in a loop from the initial conditions and the boundary conditions.

In the loop, for each value of m, the time iterator of the matrix, we set the first and last possible displacement index's to our desired boundary conditions. We then we iterate trough each displacement step and apply the central algorithm to it. We can factorise the central algorithm to slightly simplify the Matlab code:

$$\begin{aligned} U_j^{m+1} &= vU_{j-1}^m + (1-2v)U_j^m + vU_{j+1}^m \\ &= U_j^m + v(U_{j+1}^m - 2U_j^m + U_{j+1}^m) \end{aligned}$$

Where

$$v = \frac{k}{h^2}$$

Writing this in matlab as a two dimensional matrix is very similar.

```
u(m+1,j) = u(m,j) + ((k/(h^2))*(u(m,j+1) - 2*u(m,j) + u(m,j-1)));
```

Good

Physically this means that if the sum of the temperature of the two adjacent points is higher than two times the temperature of the given point, the temperature of the selected point will go up after one time step, while if the two adjacent points sum up to less than twice our current temperature, our next temperature will decrease.

As time tends to infinity the temperature will tend towards the boundary conditions , as they will set the temperature of their neighbours, which will then propagate through the curve.

Choosing h and k

Choosing h (the displacement step) and the appropriate k (time step) is not a trivial task. Picking a very small time step is computationally expensive (requires a big matrix and a computer with more memory), and we generally want to compare the temperature distribution at larger time increments (as shown in the instruction sheet).

The Von Neumann stability anlaysis[2] shows that in order to produce a stable results it is required:

$$v = \frac{k}{h^2} \leq \frac{1}{2}$$

Since we want to produce long simulations with a small distance step, but don't want to see extremely small steps in the graph, we set the time step to satisfy the equality given a value for h. Even then, the steps are hardly small so when graphing we only use a fraction of the time steps from the whole range. In our implementation we print 10 equaly spaced lines from the time range.

```
% File: ex4.m
% Coursework 17 Q4
% Using dx and dt as h and k are confusing
% h = dx
% k = dt
clear

% play about with dx to get resolution
dx = input('Input position step (like 0.01): ');

% Maximum dt to maintain stability, based on the tailor expansion.
dt = dx^2/2;

tfin = input('Final input time(like example 0.1): ');

lines = input('How many lines across the range? (like 10): ');

% Create x and t for plotting in the array
x = 0:dx:1;
t = 1:dt:tfin+1;
```

[2] https://en.wikipedia.org/wiki/Von_Neumann_stability_analysis

```
21
22  % Initialization of temperature matrix advancing in time (rows) and space (columns)
23  u = zeros(length(t),length(x));
24  u(1,:) = 0;
25  u(1,length(x)) = 0;
26
27   % Initial condition 1
28   for i = 1:length(x)
29       if x(i) <= 0.5
30           u(1,i) = 2*x(i);;
31       else
32           u(1,i) = 2*(1-x(i));
33       end
34   end
35  %
36  % % Initial condition 2
37  % for i = 1:length(x)
38  % % u(1,i) = abs(sin(2*pi*x(i)));
39  %   u(1,i) = sin(2*pi*x(i));
40  % end
41  %
42  % % Initial Condition 3
43  %  for i = 0.25/dx:0.75/dx
44  %   u(1,i) = 1;
45  %  end
46  %
47  % % Initial Condition 4
48  % for i = 1:0.25/dx
49  %   u(1,i) = -(i*dx)*4;
50  % end
51  % for i = 0.25/dx+1:0.75/dx
52  %   u(1,i) = -1+(i*dx-0.25)*4;
53  % end
54  % for i = 0.75/dx+1:length(x)
55  %   u(1,i) = 1-(i*dx)*4;
56  % end
57  %
58  %% Initial Condition 5
59  %for i = 1:0.25/dx
60  %   u(1,i) = -1;
61  %end
62  %for i = 0.25/dx+1:0.75/dx
63  %   u(1,i) = -1+(i*dx-0.25)*4;
64  %end
65  %for i = 0.75/dx+1:length(x)
66  %   u(1,i) = 1;
```

```
%end
%
%
for m = 1:length(t)
    % Set boundaries
%    u(m+1,1) = 0.5 * m /length(t);
%    u(m+1,length(x)) = 0.5 * m /length(t);
    u(m+1,1) = 0;
    u(m+1,length(x)) = 0;
    for j = 2:(length(x)-1)
        % multiply out (1-2v) and factorise out v
        u(m+1,j) = u(m,j) + ((dt/(dx^2))*(u(m,j+1) - 2*u(m,j) + u(m,j-1)));
    end
end

figure;
hold on;
j = 0
for i = 1:round(length(t)/lines):length(t)
    j = j+1;
    plot(x,u(i,:),'.');
    legendInfo{j} = ['t = ' num2str(round(i*dt, 3))];
end
legend(legendInfo,'Location', 'southeast');
xlabel('Displacement');
ylabel('Temperature');
```

Tent Function

$$y_0(x) = \begin{cases} 2x & \text{for } x \in [0, 0.5] \\ 2 - 2x & \text{for } x \in [0.5, 1] \end{cases}$$

In figure 32 we can see the tent function applied as the initial condition to a one dimensional rod of length 1. The coloured lines represent the approximated temperature at times increments of 0.1, all the way up to 1.0. Because of the boundary conditions the end points are bound to zero degrees and as a result the temperature of the rod decreases. As t approaches infinity the temperature of the rod will approach zero, where the center point which started as the highest temperature and is furthest away from the boundaries maintains the highest temperature of all points.

Sinusoidal Function

$$y_0(x) = \sin 2\pi x$$

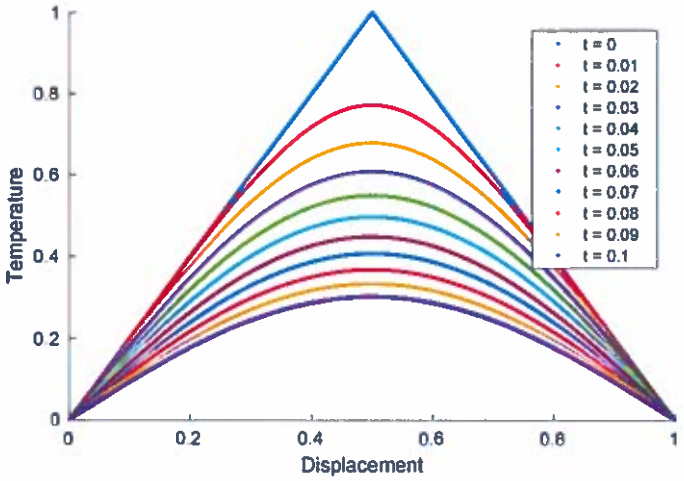


Figure 32: Tent Function

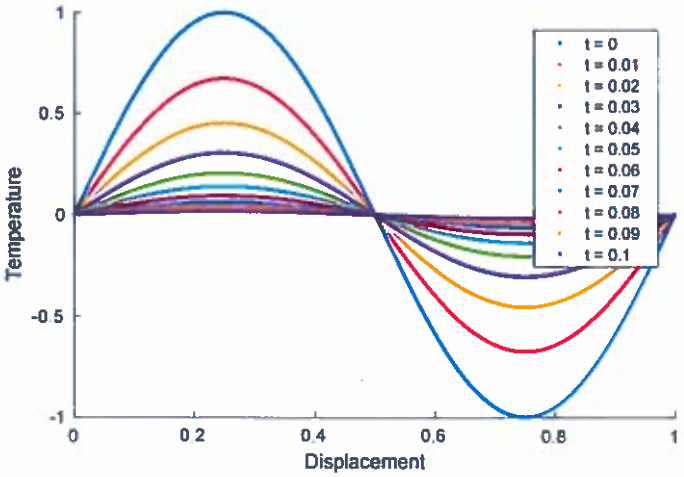


Figure 33: Sinusoidal Function A

In figure 33 we perform the finite difference method to a sinusoid. The result yet again is decreasing temperature as the boundaries are zero. The temperature decreases such that the resulting distribution continues to be sinusoidal, only with a different amplitude. The left and right side of the rod have identical but opposite amplitudes, and due to this equilibrium the center point at 0.5 displacement stays at 0 temperature.

$$y_0(x) = |\sin 2\pi x|$$

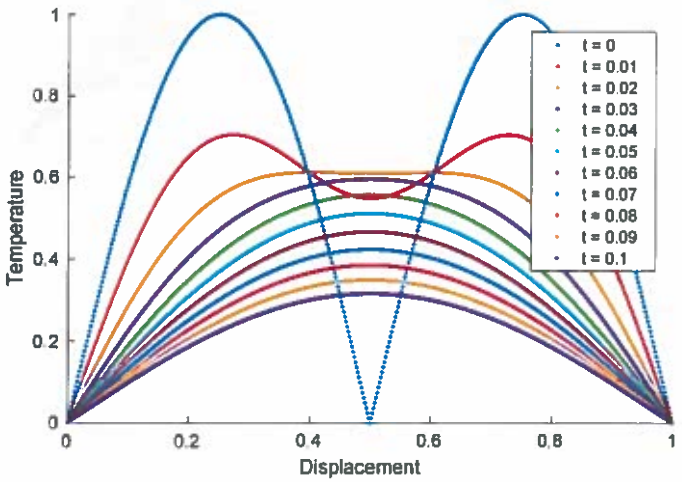


Figure 34: Sinusoidal Function B

In figure 34 we use the same initial condition function, only this time we take the absolute value of it, so what was negative before is now positive. It is interesting to note that the resulting distribution at $t \neq 0$ is very different from the absolute value of that in figure 34. Point 0.5 no longer sits at 0 and is instead heated due to neighbouring elements until it becomes the hottest point.

Square Function

$$y_0(x) = \begin{cases} 0 & \text{for } x \in [0, 0.25] \cup [0.75, 1] \\ 1 & \text{for } x \in [0.25, 0.75] \end{cases}$$

Applying the "square" function in figure 35 produces interesting results. Where as in figure 34 the center heated up while the corners cooled down the opposite happens initially, as the curve straightens out. This happens because of the extreme temperature difference that we introduced.

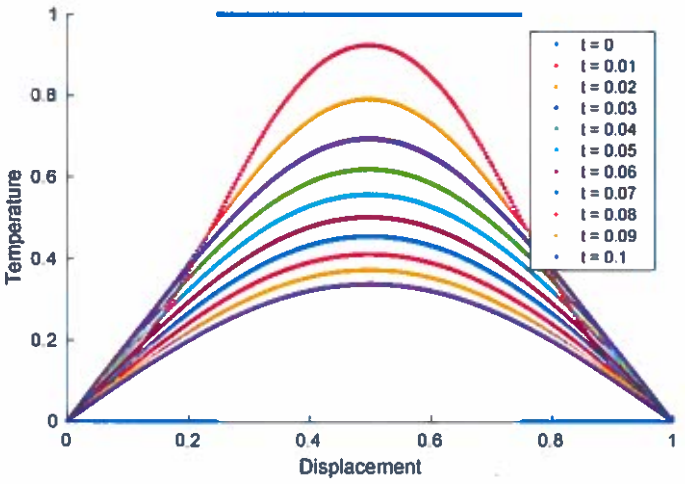


Figure 35: Square Function

Double Tent function

Making two symmetric tent functions with opposite signs like in figure 36, shows a result like that in figure 32, with an artificial boundary of zero in the centre due to the symmetry.

Varying boundary conditions

It is possible to give the algorithm initial conditions which do not match the boundary conditions. In figure 37 we can see the initial conditions in blue which attempt to set the corners of the rod to -1 and 1 respectively. As the boundaries are forced to zero they quickly pull the graph away from the original initial conditions and towards themselves. As t approaches infinity the curve in figure 37 will become a straight line between the two boundaries. Again we have an equilibrium and a constant temperature of zero in the middle.

Non Zero boundary conditions

In figure 38 we use the same initial conditions as in figure 33, however this time we set the boundary conditions to be 0.5 and 1. Yet again this forces the temperature curve towards them. The equilibrium at 0.5 displacement is no longer maintained as the boundaries are different and non-zero. As t approaches infinity we will get a straight line between 0.5 and 1 on the y-axis.

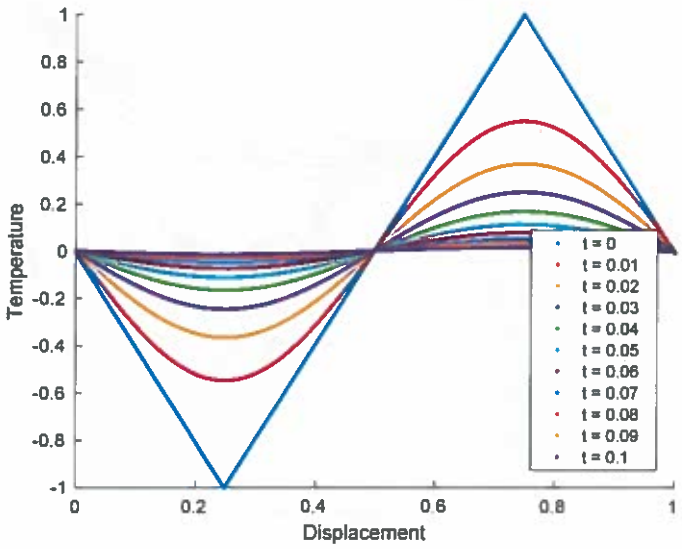


Figure 36: Double Tent

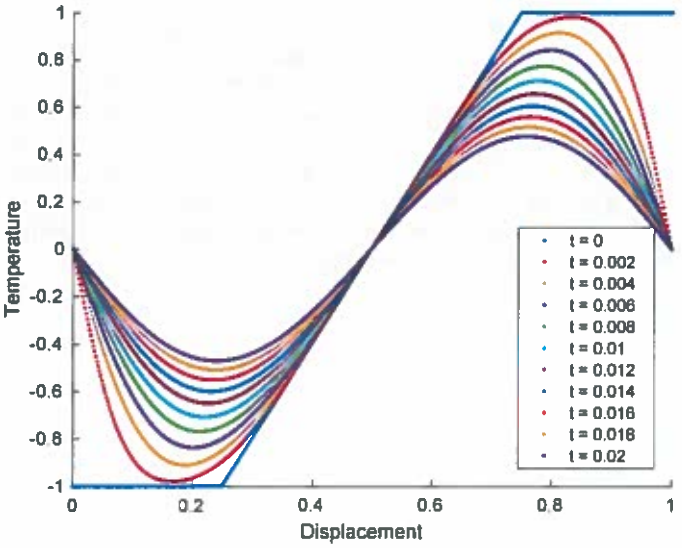


Figure 37: Non-matching initial and boundary conditions

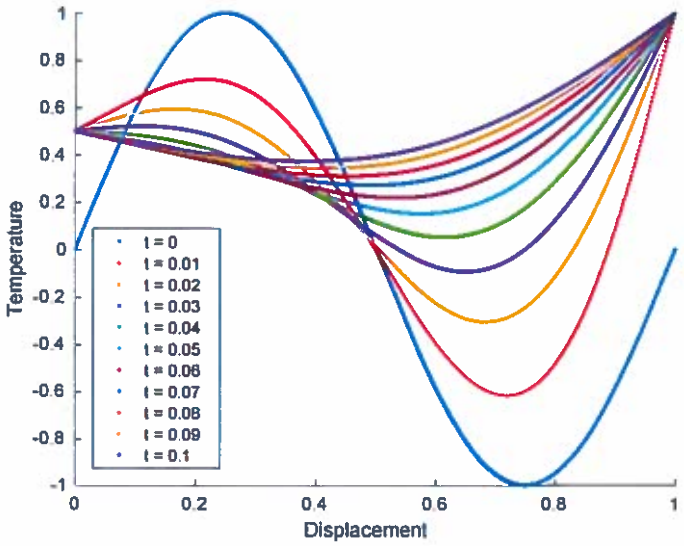


Figure 38: Non Zero Boundary Conditions

Time varying boundary conditions

Finally in every loop iteration we can change the boundary condition based on the elapsed time. We can see the effect of doing so on the original tent function in figure 39. The boundaries move from 0 degrees to 0.5 degrees at a rate of 5 degrees per second. The result is the function bending back on itself as the boundaries start moving up. It is interesting how at $t = 0$, $x = 0.5$ is the hottest point while the boundaries are the coldest points, and at $t = 0.1$, $x = 0.5$ is the coldest point while the boundaries are the hottest points. This will continue as time goes on until a non-flat equilibrium shape (with corners bending up) is reached, which goes up by 5 degrees every second.

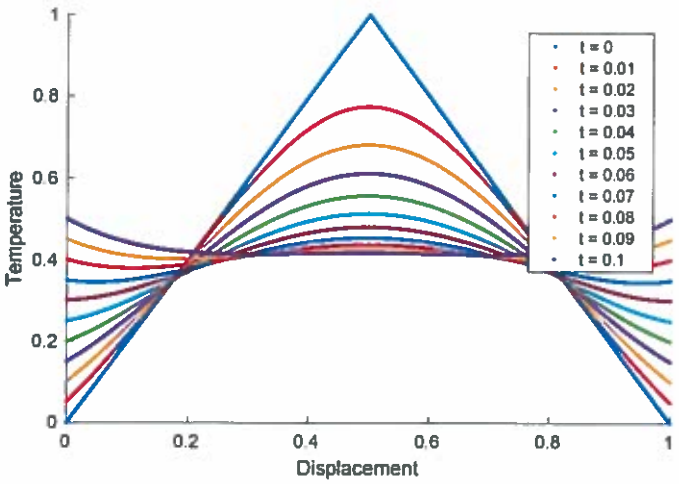


Figure 39: Varying Boundary Conditions